PROJECT REPORT

on

"Vernacular Voice-to-SOP"

By

UNDER THE GUIDANCE OF

Prof. Leena Ladge

SUBMITTED IN PARTIAL FULFILLMENT FOR THE DEGREE OF

BACHELOR OF ENGINEERING

IN INFORMATION TECHNOLOGY



Department of Information Technology

SIES Graduate School of Technology (Autonomous)

Affiliated to University of Mumbai

Nerul, Navi Mumbai - 400 706

(AY 2026-2027)

CERTIFICATE

This is to certify that this is a bonafide record of project titled **"Vernacular Voice-to-SOP: Converting Spoken Factory Procedures in Indian Languages into Structured Visual SOPs"** carried out by the following students of final year in Information Technology.

Sr. No.	Name	Roll No.
1.	Atharva Gadekar	123A3017
2.	Varun Jain	123A3022
3.	Shreya Patil	123A3036

The report is submitted in partial fulfillment of the degree course of Bachelor of Engineering in Information Technology, of University of Mumbai during the academic year 2026-27.

Internal Guide **Head of Department** **Principal**

We have examined this report as per University requirements at SIES Graduate School of Technology, Nerul (E), Navi Mumbai on _____.

Name of External Examiner: _____

Signature with Date: _____

Name of Internal Examiner: _____

Signature with Date: _____

ABSTRACT

Factory Standard Operating Procedures (SOPs) a are usually written in English, while a large share of shop-floor workers are more fluent in Hindi or Marathi, and many cannot read at all. At the same time, the hazards a worker notices first, such as a loose pedal, a missing sensor or a wrong step on the card, rarely reach anyone who can fix them. This project, Vernacular Voice-to-SOP, closes both gaps with one shared speech pipeline.

In the first feature, a supervisor narrates a procedure in Hindi or Marathi. Fine-tuned Whisper models transcribe the speech over silence-split windows, a large language model (LLM) recovers the true order of steps and flags safety hazards under a strict JSON schema, and neural text-to-speech produces audio for every step. The result is a set of numbered, picture-and-voice SOP cards that a worker can follow without reading.

In the second feature, "Bolo", a worker holds one button on the card and reports a problem or asks a question in his own words, with no login and no stored audio. Each report is compared with that machine's SOP and triaged as a machine, SOP or understanding problem, routed to the line's manager instead of the supervisor, given a 24-hour or 72-hour deadline that escalates to the plant head when missed, and clustered with similar reports. Three similar SOP complaints automatically draft a card correction, which a manager approves into the next SOP version, and the worker hears the outcome in his own language. At the client's request, the system was extended with employee, workforce and HR management modules built on the same plant and line structure and access rules.

The system is implemented with FastAPI, React and TypeScript, Supabase Postgres with Row Level Security, and Supabase Auth, and is verified with unit, integration, access-control and end-to-end tests.

Keywords: *Indian-language speech recognition, Whisper, large language models, standard operating procedures, worker safety, near-miss reporting, Hindi, Marathi.*

ACKNOWLEDGEMENT

We wish to express our deep sense of gratitude to our project guide, Prof. Leena Ladge, for providing timely assistance to our queries and guidance, and for her support as Head of Department. We also thank our Principal, Dr. K. Lakshmisudha, for her valuable guidance and immense support in providing all the necessary facilities.

We would also like to thank the entire faculty of the IT Department for their valuable ideas and timely assistance in this project. Last but not the least, we would also like to thank the teaching and non-teaching staff members of our college for their support in facilitating timely completion of this project.

Project Team

Atharva Gadekar (123A3017)
Varun Jain (123A3022)
Shreya Patil (123A3036)

CONTENTS

Sr. No.	Topic	Page No.
1	Abstract	i
2	Acknowledgement	ii
3	List of Figures and Tables	iv
4	Chapter 1: Introduction	1
	1.1 Background	1
	1.2 Need of the Project	1
	1.3 Existing System	2
	1.4 Proposed System	2
	1.5 Objectives	3
	1.6 Scope	3
	1.7 Applications	3
5	Chapter 2: Literature Survey	5
	2.1 Review of Related Work	5
	2.2 Summary of the Survey	6
	2.3 Research Gap	7
6	Chapter 3: Methodology	8
	3.1 Proposed System	8
	3.2 Drawbacks of the Existing System	9
	3.3 Problem Definition	9
	3.4 Present Report on Investigation	10
	3.5 Algorithm Implementation	11
	3.6 Technology Stack	12
7	Chapter 4: Design	14
	4.1 System Architecture	14
	4.2 Data Flow Diagrams	14
	4.3 UML Diagrams	16
	4.4 Entity-Relationship Diagram	18
	4.5 Gantt Chart	20
8	Chapter 5: Coding, Results and Discussion	21
	5.1 Implementation Overview	21
	5.2 Coding	21
	5.3 Results	23
	5.4 Testing	25
	5.5 Discussion	27
9	Chapter 6: Conclusion	28
10	References	30
11	Plagiarism Report	32

LIST OF FIGURES

Figure No.	Title	Page No.
3.1	Accident ratio (Bird, 1966) and where Bolo acts	9
3.2	Effect of ASR window length on sentence recovery	10
4.1	System architecture	14
4.2	DFD Level 0 (context diagram)	15
4.3	DFD Level 1	16
4.4	Use case diagram	17
4.5	Sequence diagram: worker voice report	18
4.6	Entity-relationship diagram	19
4.7	Gantt chart	20
5.1	Worker view: machine list and SOP cards	23
5.2	Supervisor view: recording and reviewing a procedure	24
5.3	Manager view: triaged report inbox	24
5.4	Plant head view: dashboard	24
5.5	Employee, Workforce and HR modules	25
5.6	Worker view: voice report and receipt	25

LIST OF TABLES

Table No.	Title	Page No.
1.1	Existing approaches and their limitations	2
2.1	Summary of the literature survey	6
3.1	Triage categories for worker reports	8
3.2	Technology stack	12
4.1	Main database tables	19
5.1	Backend modules and responsibilities	21
5.2	Unit and integration test cases	26
5.3	End-to-end test cases	26

CHAPTER 1
INTRODUCTION

1.1 Background

A Standard Operating Procedure (SOP) is the step-by-step instruction for running a machine or a process safely and consistently. In Indian factories these documents are usually typed in English and kept on paper or in a file near the machine. On the shop floor, however, the people who actually run the machines are frequently migrant workers who speak Hindi, Marathi or another regional language, and many have limited or no formal education. According to Census 2011, only about 10.6% of Indians speak English as a first, second or third language [14].

Supervisors already bridge this gap verbally: they explain a procedure out loud, in the worker's language. That spoken knowledge is never recorded, ordered or checked, and it is lost when the supervisor changes. In the opposite direction, the worker who stands at the machine all day is the first to notice when something is wrong, but he has no safe and simple way to report it. This project uses speech recognition and language models to carry knowledge in both directions, in the languages the shop floor actually speaks.

1.2 Need of the Project

Industrial safety data for India shows the cost of this gap:

- **Fatalities:** DGFASLI data for 2017–2020 shows that on average three workers died and eleven were injured every day in India's registered factories [12].
- **Hand injuries on presses:** In 2025, Safe in India assisted 1,729 injured auto-sector workers. 93% of these injuries involved power presses with no safety sensors, and 72% were crush injuries that typically cost about two fingers [11], [13].
- **No training:** 37% of injured workers learned to operate the machine while working on it; most received no formal training [13].
- **Migrant workforce:** 85% of injured workers in Haryana and 63% in Maharashtra were migrants, mainly from Bihar, Uttar Pradesh and Madhya Pradesh [13].

- **Under-reporting:** In Haryana in 2023, Safe in India alone assisted 1,041 injured factory workers, while the official DGFASLI record for the whole state showed 40 non-fatal injuries [13].

Safety research also explains why warnings go unheard. Bird's analysis of 1.7 million accident reports found roughly 600 near-misses for every serious injury [15]. Workers are the first to see these near-misses, but studies show that supervisor behaviour strongly predicts how many accidents go unreported [6], and that employees see leaders several levels above them as part of the reason speaking up feels risky or futile [7]. A system is therefore needed that (a) gives workers instructions they can understand without reading, and (b) gives workers a safe, simple channel to report hazards that bypasses the supervisor and guarantees follow-up.

1.3 Existing System

Table 1.1 summarises how SOPs are written and how worker complaints are handled today.

Table 1.1: Existing approaches and their limitations

Existing approach	Limitation
Manual SOP authoring	A supervisor writes procedures in English by hand; slow, and rarely updated after process changes.
Generic cloud ASR (Google, Azure, AWS)	Paid per minute, needs internet, and weak on accented Indic speech in noisy plants.
Indic ASR research systems (IndicWhisper, Vistaar)	Strong transcription, but stop at plain text with no structure or safety tagging.
Worker complaints (verbal, paper forms, suggestion boxes)	Unusable for workers who cannot write, and filtered by the supervisor closest to the problem.
Voice incident-reporting apps (including Hindi/Marathi)	Built for literate safety officers filling forms, not for shop-floor workers.

As a result, no existing system produces audio-narrated vernacular SOPs, and none lets a non-literate worker report a problem safely and know that it was heard.

1.4 Proposed System

The proposed system, Vernacular Voice-to-SOP, provides two connected features built on one shared speech pipeline (speech recognition, then an LLM, then text-to-speech), plus a set of plant management modules requested by the client:

- **Feature 1 – Voice-to-SOP:** the supervisor narrates a procedure in Hindi or Marathi; the system transcribes it, orders the steps, flags hazards, generates per-step audio and publishes visual SOP cards.
- **Feature 2 – Worker Voice Feedback ("Bolo"):** the worker holds one mic button on the card and reports a problem or asks a question in his own words. The report is triaged against the SOP, routed anonymously to the manager above the supervisor with a deadline and escalation, clustered with similar reports, and answered back in the worker's language. Repeated complaints about a card draft a correction for the manager to approve.
- **Feature 3 – Plant management modules:** this project is being developed for an outside client, who asked for employee, workforce (shifts and attendance) and HR (leave and recruitment) management. These modules reuse the same plant and line structure and the same two-layer authorisation as Features 1 and 2.

1.5 Objectives

1. To achieve reliable Hindi and Marathi transcription of spontaneous, unscripted shop-floor speech.
2. To recover the true logical order of steps from unordered speech and automatically flag safety hazards.
3. To deliver each SOP as numbered visual cards with vernacular audio for low-literacy workers.
4. To classify each worker report as a machine, SOP or understanding problem by comparing it with the SOP.
5. To protect worker identity: no stored audio, no name and no worker login.
6. To guarantee follow-up through skip-level routing, deadline-based escalation and a spoken reply to the worker.
7. To support plant operations (employees, shifts, attendance, leave and recruitment) under the same access model, as required by the client.

1.6 Scope

The project covers the following:

- Speech input in Hindi and Marathi, using fine-tuned Whisper checkpoints for each language and automatic language identification.
- SOP creation, versioning and playback for machines organised by plant and production line.
- Anonymous worker reports and questions through the web app on a shared phone or tablet at the machine.
- Role-based staff access for supervisors, managers, plant heads, HR administrators and recruiters.
- Employee directory, shift scheduling, supervisor-operated attendance, leave requests with approval, and a minimal recruitment pipeline.

Outside the current scope are a phone-call (IVR) channel for basic phones, languages other than Hindi and Marathi, payroll, and a validated pilot in a working plant; these are listed as future work in Chapter 6.

1.7 Applications

- Digitising machine operating procedures on manufacturing shop floors without a technical writer.
- Onboarding and training new or migrant workers in the language they actually speak.
- Anonymous hazard and near-miss reporting for non-literate workers, catching worn pedals, missing guards and wrong cards before an injury.
- Continuous SOP improvement, as confirmed worker reports become reviewed card corrections.
- Safety compliance and audit: hazard steps, report deadlines and escalations are logged per line and machine.
- Training-needs analysis from "understanding" reports, which point to steps workers find confusing.
- Plant-wide employee directory, shift scheduling and attendance, replacing paper registers and verbal rosters.
- Transferable to agriculture advisories, healthcare protocols and rural skill training with only a vocabulary change.

CHAPTER 2
LITERATURE SURVEY

The literature relevant to this project falls into four groups: speech recognition for Indian languages, reliable structured output from language models, voice interfaces for factory and low-literacy users, and the organisational causes of safety under-reporting. Each paper is reviewed below for its findings and its gap with respect to this project.

2.1 Review of Related Work

Speech recognition for Indian languages. Bhogale et al. [1] fine-tuned Whisper [10] into IndicWhisper on 10.7K hours of speech across 12 Indian languages and achieved the best word error rate on 39 of 59 benchmarks. A PReMI study [2] benchmarked several Whisper sizes on 15 Indic languages and found that Hindi and Tamil perform well while several low-resource languages fail, showing that per-language accuracy cannot be assumed from an aggregate score. A 2025 preprint [3] used reinforcement learning to detect Hindi-Marathi code-switching while fine-tuning Whisper-small. All three stop at a transcript: none orders steps, tags hazards or produces an artefact a worker can use.

Reliable structured output from language models. SLOT [4] adds a fine-tuned lightweight model after the LLM to force schema-valid JSON and reaches 99.5% schema accuracy. The cost is a second trained model in the loop, which is too heavy for offline, low-cost deployment on factory hardware. This project instead validates the LLM's reply against a strict schema and retries once.

Voice interfaces for factories and low-literacy users. Longo and Padovano [5] proposed a framework for voice assistants in operator workflows and identified noisy, multilingual shop floors as a core challenge, but did not build a working system and assumed English. CGNet Swara [8] showed that a phone-based voice forum can serve rural, low-literacy users, receiving more than 70,000 calls in 21 months, but every message was reviewed by human moderators with no automatic transcription, classification or routing.

Safety reporting and under-reporting. Probst [6] surveyed 1,238 employees in 33 organisations and found that supervisor enforcement behaviour predicted how many accidents went unreported. Detert and Treviño [7] found that employees blamed leaders two to five levels above them almost as often as their immediate boss when speaking up

felt risky or futile. Tixier et al. [9] built an NLP system that extracts 101 attributes from written injury reports with over 95% accuracy, but it works only on English, after-the-fact reports and never compares a report with the SOP. None of these provides a channel that bypasses the supervisor for workers who cannot write.

2.2 Summary of the Survey

Table 2.1: Summary of the literature survey

Sr.	Author	Key findings	Gap
1	K. S. Bhogale, T. Javed, M. M. Khapra et al. [1]	Fine-tunes Whisper into IndicWhisper on 10.7K hours across 12 Indian languages; best WER on 39 of 59 benchmarks, with an average WER reduction of 4.1.	Output ends at a raw transcript, with no step structuring or safety tagging; Marathi is averaged into a 12-language score.
2	PReMI proceedings [2]	Benchmarks Whisper medium, turbo and large on 15 Indic languages using WER, CER and PER; Hindi and Tamil perform best and several low-resource languages fail.	A benchmarking study only, with no application layer; it shows that per-language accuracy cannot be assumed from an aggregate score.
3	Preprints.org 202503.2278 [3]	Uses Q-Learning and Deep Q-Networks for dynamic language-switch detection while fine-tuning Whisper-small on Common Voice 13.0.	Switch detection is the end goal in isolation; no downstream task consumes the transcript, so no usable artefact is produced.
4	D. Y.-B. Wang, Z. Shen, Z. Xu, H. Ding et al. [4]	Adds a fine-tuned lightweight model as a post-processing layer to force schema-valid JSON, reaching 99.5% schema accuracy.	Reliability costs a second trained model in the loop, which is too heavy for offline, low-cost deployment on factory hardware.
5	F. Longo, A. Padovano [5]	A Cognitive Work Analysis framework for where voice assistants fit in factory operator workflows; flags noisy, multilingual floors as a core challenge.	Conceptual and English-centric: a design framework, not a working system, with no regional-language support.
6	T. M. Probst [6]	A survey of 1,238 employees in 33 organisations found that supervisor enforcement behaviour predicted how many accidents employees left unreported; the effect weakened under a positive safety climate.	Explains why workers stay silent, but offers no reporting channel that bypasses the supervisor.
7	J. R. Detert, L. K. Treviño [7]	Employees cited leaders two to five levels above them almost as often as their immediate boss when explaining why speaking up felt risky or futile.	An organisational study of literate employees, with no channel for low-literacy workers and no mechanism that guarantees follow-up.
8	P. Mudliar, J. Donner, W. Thies [8]	Callers record reports from any basic phone; the forum received 70,000+ calls and published 1,100 messages in its first 21 months.	Every message is reviewed by human moderators, with no automatic transcription, classification or routing; not built for workplace safety.
9	A. J.-P. Tixier, M. R. Hallowell et al. [9]	An NLP system scans unstructured injury reports for 101 attributes and outcomes with over 95% accuracy.	Works on English, written, after-the-fact injury reports; never checks a report against the SOP and takes no voice input.

2.3 Research Gap

- No reviewed work carries vernacular speech all the way to a usable, ordered SOP artefact; hazard detection from speech remains conceptual.
- Voice channels for low-literacy users exist, but rely on human moderators; nothing triages a spoken report automatically.
- Safety-text NLP never compares a report with the SOP, so it cannot tell a machine fault from a wrong card or a training gap.
- Under-reporting research names the supervisor as the barrier, yet no system routes reports past the supervisor with anonymity and guaranteed follow-up.

This project addresses all four gaps in a single system.

CHAPTER 3
METHODOLOGY

3.1 Proposed System

The proposed system is a single web application with two connected features on one shared speech stack, extended with plant management modules. Its architecture is shown in Figure 4.1 (Chapter 4).

Feature 1 – Voice-to-SOP (supervisor to worker). The supervisor records or uploads a spoken explanation of a procedure. The audio is transcribed by a fine-tuned Whisper model for the detected language (Stage 1), structured by an LLM into ordered, atomic steps with a safety flag and an icon (Stage 2), voiced step by step with a neural Hindi or Marathi voice (Stage 3), and presented as numbered visual cards (Stage 4). Every save creates a new SOP version for that machine.

Feature 2 – Bolo (worker to manager). The worker holds the mic button on the card he is using. The same ASR transcribes the clip, after which the audio file is deleted. The LLM then compares the text with that machine's SOP and returns a kind, a severity, a one-paragraph summary and the step it concerns (Table 3.1). The report is routed to the manager of the line, never the supervisor, with a deadline of 24 hours for a high-severity machine fault and 72 hours otherwise. A report that is still open after its deadline escalates to the plant head. Open reports on the same machine, kind and step within 30 days are clustered, and a cluster of three is "confirmed". A confirmed SOP cluster drafts a card edit, which the manager approves into the next SOP version. A worker question that the SOP already answers is answered back aloud and never enters the manager's inbox.

Table 3.1: Triage categories for worker reports

Kind	Meaning	Example and outcome
machine	The equipment is faulty or unsafe; the SOP is fine.	"The pedal is loose" – urgent ticket for the manager and maintenance.
sop	The card is wrong or missing something.	"The card says 15 mm, the bolt is 17 mm" – suggested card change.
understanding	The SOP is correct but the worker is confused by it.	Steps done in the wrong order – flagged as a training need.

Feature 3 – Plant management modules. Added at the client's request: a plant-wide employee directory with departments and an org chart; shift definitions, line-scoped shift assignments and supervisor-operated (kiosk-style) clock-in/out; self-service leave requests with balances and approval; and a minimal recruitment pipeline (job postings, candidates, applications) with resumes stored in cloud storage. Line workers are tracked by name for scheduling but are never given an account.

Figure 3.1 shows why the worker channel matters for safety. For every serious injury there are about 600 near-misses [15]; Bolo is designed to capture that bottom layer, when a hazard is visible but no one has been hurt yet.

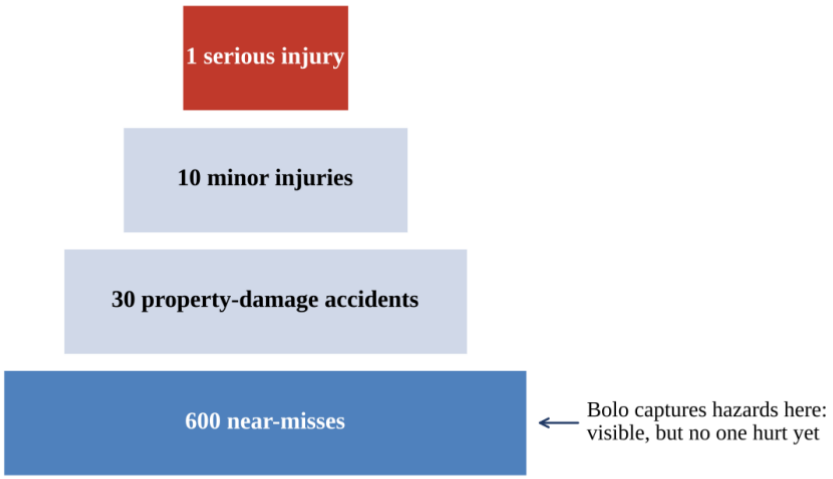


Figure 3.1: Accident ratio (Bird, 1966) and where Bolo acts

3.2 Drawbacks of the Existing System

- SOPs are in English text, which a large part of the workforce cannot read.
- Converting a supervisor's spoken explanation into a written SOP is manual, slow and rarely repeated after process changes.
- General-purpose cloud ASR is paid, needs internet and handles accented, noisy Indic speech poorly; research ASR stops at raw text.
- Worker complaints depend on writing or on the supervisor, who may filter or ignore them; there is no anonymity and no deadline.
- Existing reporting tools cannot tell whether a problem lies with the machine, the card or the training, because they do not know what the SOP says.
- Workers never hear back, so they stop reporting.

3.3 Problem Definition

To design and implement a system that converts an unstructured spoken factory procedure in Hindi or Marathi into an ordered, safety-annotated visual SOP with per-step audio, and that lets non-literate workers report machine or SOP problems by voice – anonymously, triaged against the SOP, routed past the supervisor, and acknowledged in their own language.

3.4 Present Report on Investigation

Choice of ASR checkpoints. General multilingual Whisper models are uneven across Indian languages [2], so the system uses a separately fine-tuned checkpoint per language: vasista22/whisper-hindi-medium for Hindi and steja/whisper-small-marathi for Marathi, with openai/whisper-tiny used only for language identification. Candidate checkpoints were to be benchmarked on AI4Bharat's Kathbath and MUCS test sets with WER and CER [16], [17] before the pipeline was wired.

Long-form truncation. These fine-tunes were trained on short utterance-level clips. On procedures longer than about 30 seconds, both chunked pipeline decoding and native long-form generation silently dropped or garbled the middle of the audio, and fixed-length windows cut words in half. The audio is therefore split only at silence gaps (librosa, 30 dB threshold) and grouped into windows of at most a few seconds. The window length was swept against clips with known text (Figure 3.2): a 68-second, 15-sentence clip recovered all 15 sentences at 4 s and 5 s windows, 14 at 6 s, 10 at 8 s and only 4 at 25 s. A 4-second maximum was chosen, which also keeps all three sentences of a typical 10-second worker complaint. A guard logs any window whose output falls below 3 characters per second of audio, which indicates that the model stopped early.

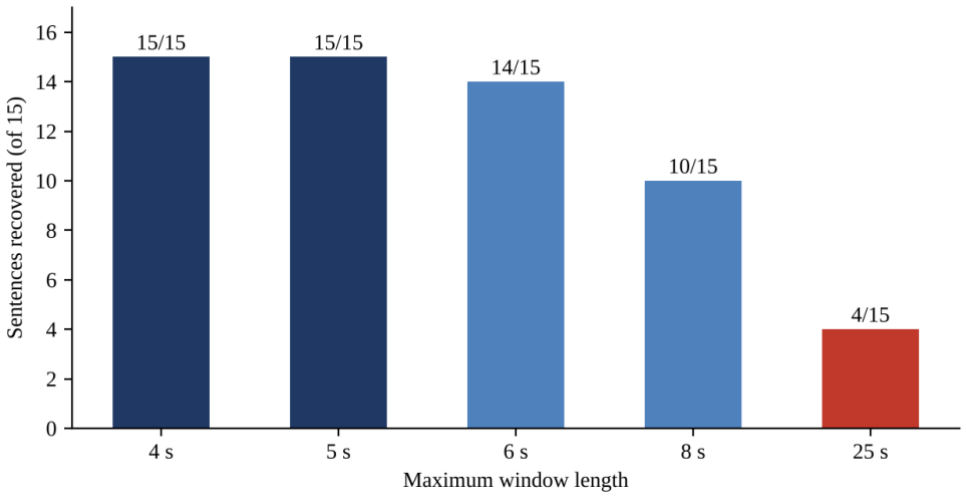


Figure 3.2: Effect of ASR window length on sentence recovery

Generation configuration. The fine-tuned checkpoints ship an outdated generation configuration that crashes on current versions of the transformers library. Because they share the architecture of the base OpenAI models, the base model's generation configuration (whisper-medium for Hindi, whisper-small for Marathi) is loaded in its place without touching the weights.

Reliable structured output. Instead of a second trained model [4], the LLM is given a fixed system prompt with an explicit JSON schema and a low temperature (0.2). Its reply is parsed (with or without a markdown code fence) and validated with Pydantic; on failure, the model is asked once more with a corrective nudge. Any step reference the model returns that is not one of the SOP's own steps is discarded.

Privacy by design. Report audio is deleted immediately after transcription and is never fetchable, so a supervisor cannot recognise a voice. With the default local ASR backend, audio never leaves the host; an optional remote backend is faster but sends the clip to the transcription provider. Workers receive an unguessable receipt instead of an account.

Routing past the supervisor. Following the under-reporting findings [6], [7], reports are assigned to the line's manager and are invisible to supervisors. Escalation and cluster size are computed when the inbox is read, so there is no background scheduler that can silently stop.

3.5 Algorithm Implementation

Algorithm 1: Silence-split transcription

```
Input: audio file, optional language hint
Output: transcript text, detected language
1. Load audio at 16 kHz mono
2. If no hint: language <- whisper-tiny language ID (fallback "hi")
3. intervals <- non-silent intervals (top_db = 30)
4. Group consecutive intervals into windows of at most 4 s,
   splitting only at silence gaps
5. For each window: text_i <- fine-tuned Whisper(language)
   warn if len(text_i) / duration < 3 chars per second
6. Return join(text_i), language
```

Algorithm 2: SOP structuring with schema validation

```
Input: transcript
Output: steps[] = {step_number, text, is_safety_warning, icon}
1. messages <- [system prompt with JSON schema, transcript]
2. reply <- LLM(messages, temperature = 0.2)
3. Try: parse JSON (strip code fence) and validate schema
4. On failure: append reply + "return ONLY valid JSON" nudge;
   reply <- LLM(messages); parse and validate (raise if invalid)
5. Save steps as version (latest + 1) of the machine's SOP
```

Algorithm 3: Per-step speech synthesis with caching

```
Input: step text, language
Output: cache key for an MP3 clip
1. voice <- hi-IN-SwaraNeural or mr-IN-AarohiNeural
2. key <- SHA-256(voice + "::" + text)
3. If audio_cache/key.mp3 does not exist: synthesize with edge-tts and save
4. Return key
```

Algorithm 4: Worker report processing (Bolo)

```
Input: audio, sop_id, optional step_id (no identity)
1. Save audio; insert report(status = received, receipt = random 16-byte
token)
2. Return 202 with receipt immediately; continue in background:
3.   text, lang <- Algorithm 1(audio); delete audio file (always)
4.   kind, severity, summary, step, suggestion <- LLM triage(SOP steps, text)
5.   assigned_to <- manager of the machine's line
6.   deadline <- created_at + (24 h if kind = machine and severity = high
                               else 72 h)
7.   Cluster: if an open report exists on the same machine, kind and step
     within 30 days, give both the same cluster_id
8.   If cluster size >= 3 and kind = sop: draft a step edit from
     the latest suggested change
9. On any error: status <- failed (worker still holds the receipt)
```

Algorithm 5: Answering a question from the SOP

```
1. text, lang <- Algorithm 1(audio); delete audio
2. covered, answer, step <- LLM(SOP cards, text)
3. If covered: speak answer via Algorithm 3; mark resolved and
answered_by_sop
   (never shown in any manager inbox)
4. Else: process as a report (Algorithm 4, steps 4-8)
```

Algorithm 6: Escalation and approval of a card correction

```
On inbox read:
  escalated <- status = open AND now() > deadline
  effective_assignee <- plant head if escalated else line manager
  confirmed <- cluster size >= 3; sort confirmed clusters first
On manager approval of a step edit:
  1. new SOP <- copy of latest version with version + 1
  2. copy all steps, replacing the edited step's text
  3. mark the edit approved; resolve every report in the cluster
  4. old version kept for audit
```

3.6 Technology Stack

Table 3.2: Technology stack

Layer	Technology
Speech recognition	Hugging Face transformers, fine-tuned Whisper (Hindi medium, Marathi small), whisper-tiny for language ID, librosa
Language model	Any OpenAI-compatible endpoint: Ollama (llama3.2:3b) offline, Groq when hosted
Text-to-speech	edge-tts neural voices hi-IN-SwaraNeural and mr-IN-AarohiNeural
Backend	Python, FastAPI, Pydantic v2, SQLAlchemy Core (raw SQL), Alembic migrations, SlowAPI rate limiting
Database and auth	Supabase Postgres with Row Level Security, Supabase Auth (JWT verified against JWKS), Supabase Storage
Frontend	React, Vite, TypeScript, Tailwind CSS
Quality and operations	pytest, ruff, GitHub Actions CI, Docker, structured JSON logs, Prometheus metrics

CHAPTER 4
DESIGN

4.1 System Architecture

Figure 4.1 shows the overall architecture. Three kinds of client share one React application: the worker view (no login), the supervisor view, and the manager, plant head and HR views. All requests reach a FastAPI backend whose modules are independently testable: asr.py, structuring.py and tts.py form the shared speech pipeline; triage.py, guidance.py, routing.py and corrections.py implement the worker feedback loop. The LLM is reached through an OpenAI-compatible endpoint, so the same code runs offline with Ollama or hosted with Groq. Data lives in Supabase Postgres, where Row Level Security enforces the same line and plant scope that the application checks.

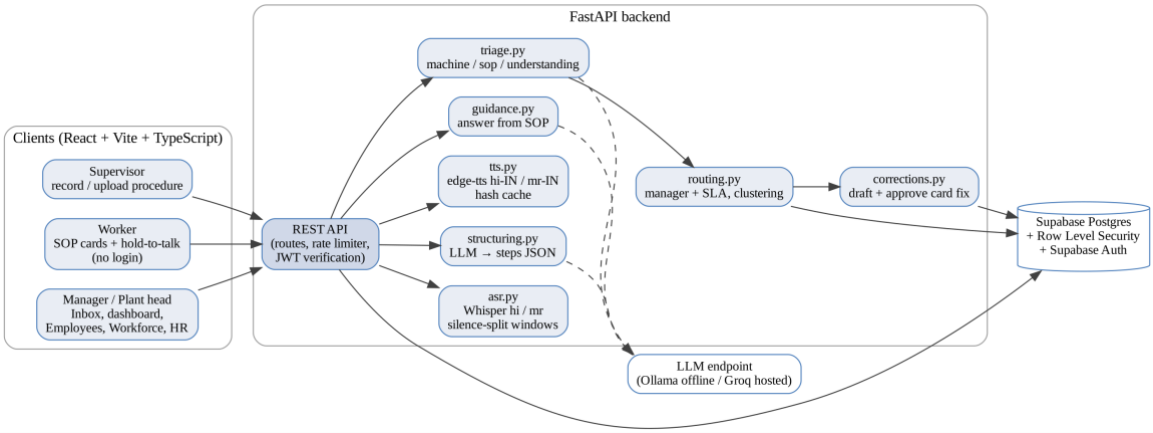


Figure 4.1: System architecture

4.2 Data Flow Diagrams

The context diagram (Figure 4.2) shows the system with its four external entities. The supervisor supplies spoken procedures; the worker receives cards and audio and supplies voice reports; the manager receives triaged reports and returns replies and approvals; the plant head and HR administrator use the dashboard and the plant management modules.

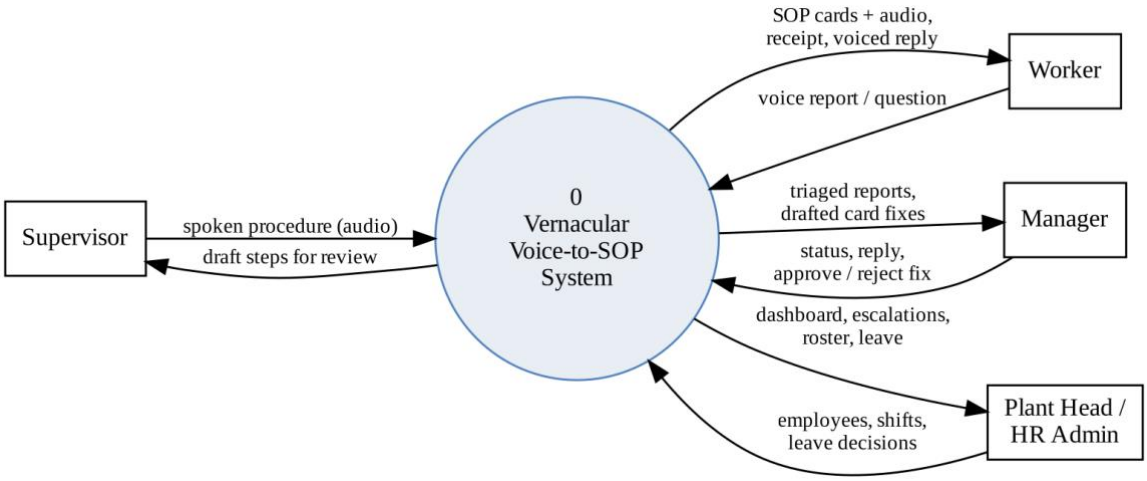


Figure 4.2: DFD Level 0 (context diagram)

Figure 4.3 expands the system into six processes and four data stores. A procedure flows through transcription (1.0), structuring (2.0) and speech synthesis (3.0) into the SOP and audio stores. A worker report flows through transcription (1.0, with the audio deleted), triage against the SOP (4.0) and routing (5.0) to the manager; confirmed SOP clusters feed the correction process (6.0), which writes the next SOP version.

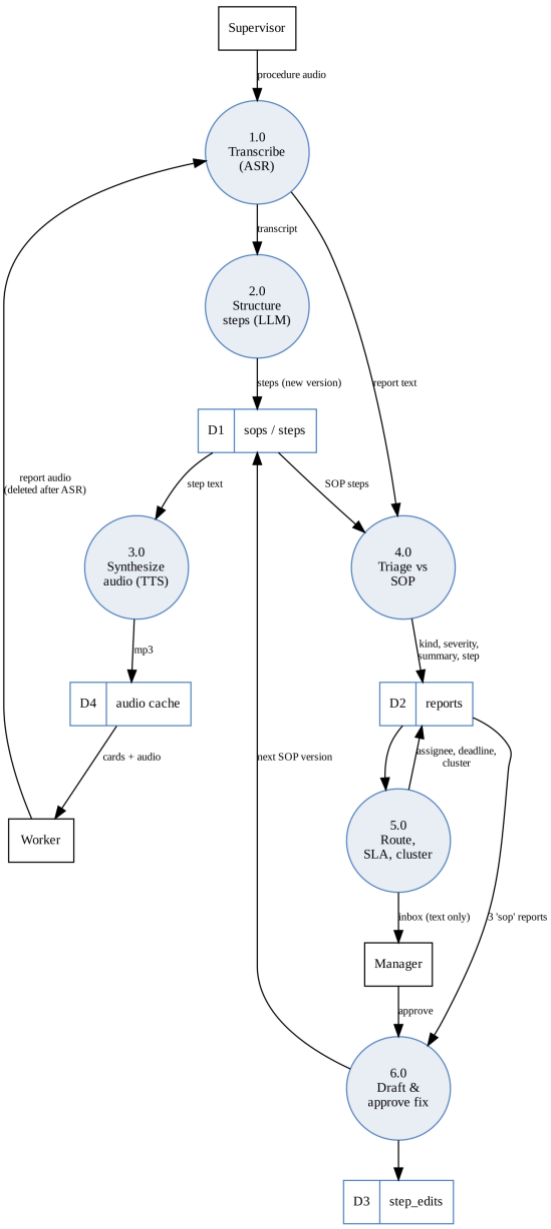


Figure 4.3: DFD Level 1

4.3 UML Diagrams

Figure 4.4 shows the use cases for each actor. The worker has no account but can view cards, report, ask and hear replies; staff roles are separated so that supervisors author SOPs but cannot see reports, managers handle their own lines' reports, and the plant head sees the whole plant.

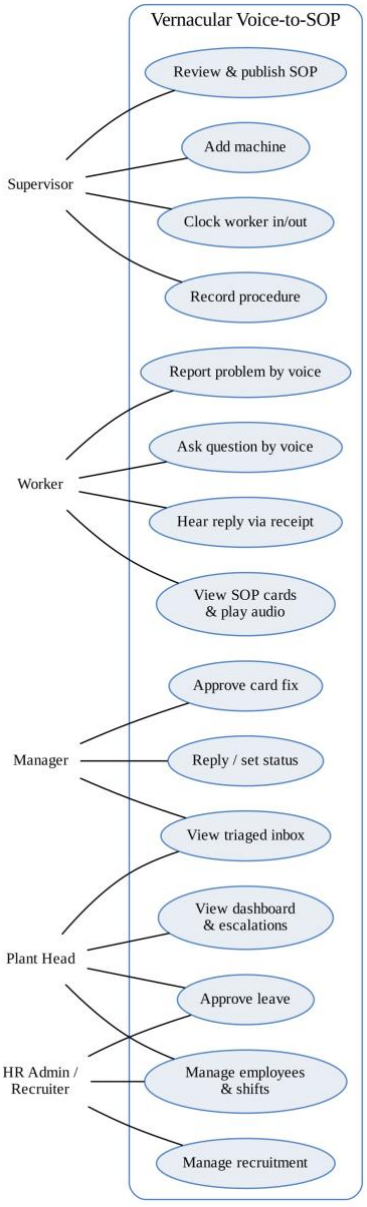


Figure 4.4: Use case diagram

Figure 4.5 shows the sequence of a worker voice report. The API answers at once with a receipt, while transcription, deletion of the audio, triage and routing run as a background task. The manager later reads the report and replies, and the reply is voiced and played back to the worker through the receipt.

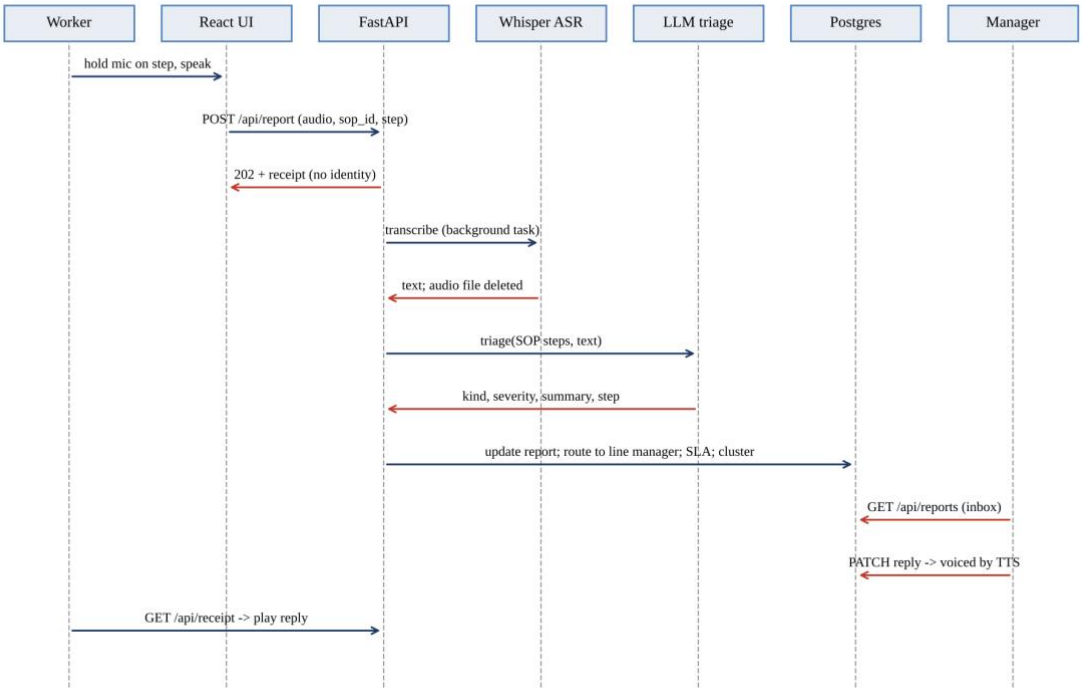


Figure 4.5: Sequence diagram: worker voice report

4.4 Entity-Relationship Diagram

Figure 4.6 shows the database design. The core chain plant – line – machine – SOP – step carries the SOP content; reports and step edits implement the feedback loop (shown in red); departments, shifts, attendance, leave and recruitment implement the client's plant management modules (shown in grey). Every table that staff touch is protected by Row Level Security.

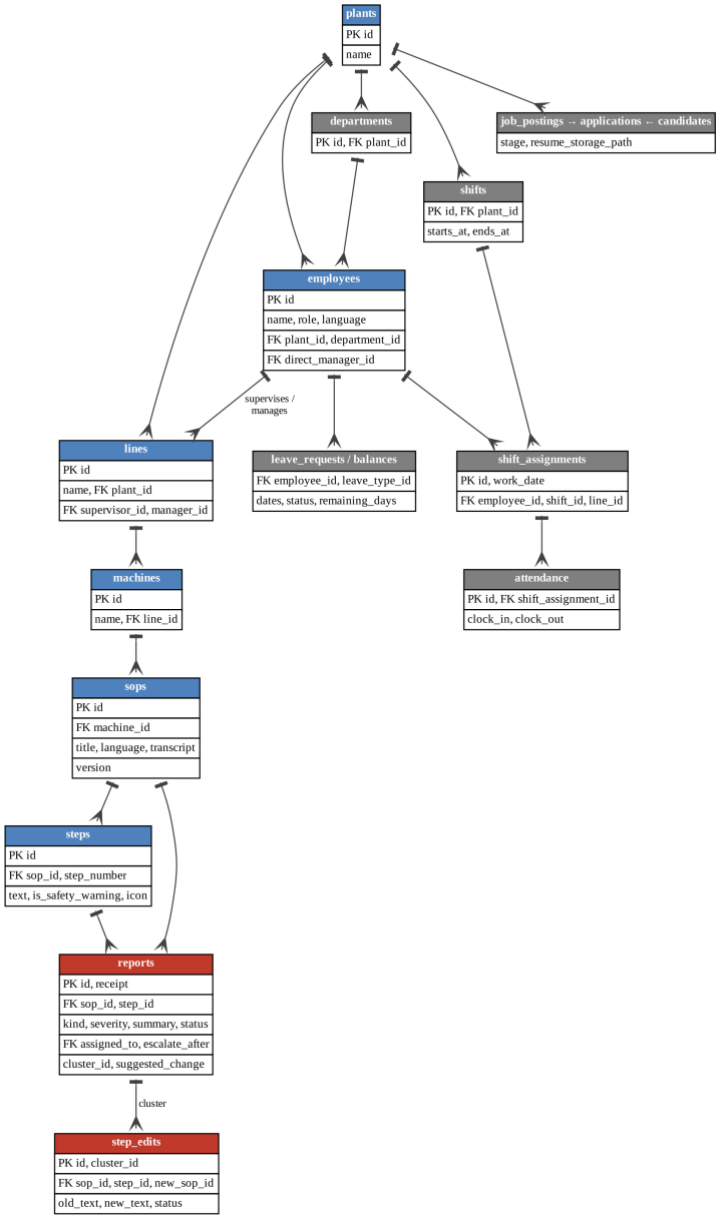


Figure 4.6: Entity-relationship diagram

Table 4.1: Main database tables

Table	Purpose
plants, lines, machines	Organisation structure; each line has one supervisor and one manager.
employees	Staff and line workers with role, language, plant, department and direct manager.
sops, steps	Versioned SOPs per machine; each step has text, a safety flag and an icon.
reports	Worker reports: receipt, kind, severity, summary, step, assignee, deadline, cluster.
step_edits	One drafted card correction per confirmed SOP cluster, with approval status.
shifts, shift_assignments, attendance	Shift definitions, daily line assignments and supervisor-operated clock-in/out.
leave_types, leave_balances, leave_requests	Leave quotas, balances and requests with approval.
job_postings, candidates, applications	Minimal recruitment pipeline with stages and resume storage paths.

4.5 Gantt Chart

Figure 4.7 shows the project schedule for Semester VII. Implementation milestones follow the project repository history: Feature 1 and deployment in early August, Feature 2 in September, then staff authentication, the React interface, the Supabase migration and the client's modules in early October.

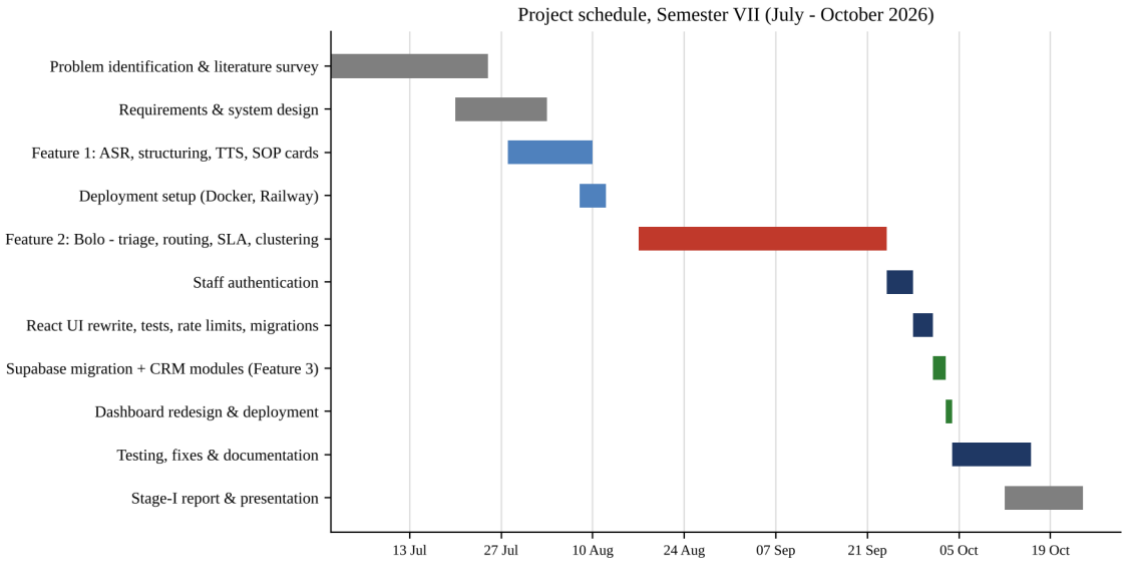


Figure 4.7: Gantt chart

CHAPTER 5
CODING, RESULTS AND DISCUSSION

5.1 Implementation Overview

Both features ship as one FastAPI and React application in a single Docker image. The backend is organised as small modules with one responsibility each (Table 5.1). The database schema is managed by hand-written Alembic migrations, including the Row Level Security policies; nothing is created automatically on start-up.

Table 5.1: Backend modules and responsibilities

Module	Responsibility
asr.py	Language ID, silence-split windowing, per-language Whisper transcription, optional remote ASR
structuring.py, llm.py	Transcript to steps JSON; schema-validated LLM calls with one retry
tts.py	Per-step speech synthesis with a content-hash disk cache
triage.py	Classifies a report as machine, sop or understanding against the SOP
guidance.py	Answers a worker question from the SOP, or hands it to a manager
routing.py	Assigns the line manager, sets the deadline, clusters similar reports
corrections.py	Drafts a step edit from a confirmed cluster; approval writes the next SOP version
auth.py	Verifies Supabase JWTs and applies line and plant scope to every query
routes/	REST endpoints for SOPs, machines, reports, employees, workforce, leave and recruitment

5.2 Coding

The following excerpts are taken from the project source code.

(a) LLM structuring prompt (app/structuring.py)

```
SYSTEM_PROMPT = """You are converting a factory manager's spoken,
unstructured
procedure into a clean SOP. The manager may explain steps out of order or
repeat things.
1. Identify the TRUE logical sequence of steps, regardless of the order
   they were mentioned in.
2. Break it into numbered, atomic steps (one action per step).
3. For each step, detect if it mentions a safety hazard (heat, sharp edges,
   electrical, chemicals, moving parts, or explicit warnings like 'be
   careful'). If yes, set is_safety_warning: true.
4. Suggest a simple icon keyword for each step.
5. Keep step text in the ORIGINAL language the manager used.
Output strict JSON only, in this schema:
{ "steps": [ { "step_number": int, "text": string,
  "is_safety_warning": bool, "icon": string } ] }"""
```

(b) Schema-validated LLM call with one retry (app/llm.py)

```
async def call_json[T: BaseModel](messages: list[dict], model_cls: type[T]) -
> T:
    content = await _call_llm(messages)
    try:
        return model_cls.model_validate(_extract_json(content))
    except (json.JSONDecodeError, ValidationError):
        pass
    retry = [*messages, {"role": "assistant", "content": content},
             {"role": "user", "content": _RETRY_NUDGE}]
    content = await _call_llm(retry)
    return model_cls.model_validate(_extract_json(content))
```

(c) Splitting audio only at silence (app/asr.py)

```
def _split_on_silence(audio: np.ndarray) -> list[np.ndarray]:
    intervals = librosa.effects.split(audio, top_db=_SILENCE_TOP_DB)
    if len(intervals) == 0:
        return [audio]
    windows = []
    cur_start, cur_end = intervals[0]
    for start, end in intervals[1:]:
        if (end - cur_start) / _SAMPLE_RATE > _WINDOW_SECONDS:
            windows.append(audio[cur_start:cur_end])
            cur_start = start
        cur_end = end
    windows.append(audio[cur_start:cur_end])
    return windows
```

(d) Skip-level routing with a deadline (app/routing.py)

```
UPDATE reports SET
    assigned_to = (SELECT l.manager_id FROM sops s
                   JOIN machines m ON m.id = s.machine_id
                   JOIN lines l ON l.id = m.line_id
                   WHERE s.id = reports.sop_id),
    escalate_after = created_at + make_interval(hours =>
        CASE WHEN kind = 'machine' AND severity = 'high'
             THEN :urgent ELSE :default END)      -- 24 h / 72 h
WHERE id = :id
```

(e) Accepting an anonymous report (app/routes/report.py)

```
@router.post("/report", response_model=ReportCreated, status_code=202)
@limiter.limit("20/minute")
def create_report(request, background, audio: UploadFile = File(...),
                  sop_id: int = Form(...), step_id: int | None = Form(None),
                  language: str = Form("auto")) -> ReportCreated:
    ...
    receipt = secrets.token_urlsafe(16)
    report_id = conn.execute(text("INSERT INTO reports (sop_id, receipt) "
        "VALUES (:sop_id, :receipt) RETURNING id"), {...}).scalar_one()
    background.add_task(process_report, report_id, audio_path, sop_id,
                        step_id, language)
    return ReportCreated(report_id=report_id, receipt=receipt,
status="received")
```

5.3 Results

The screens below show the system running end to end, in the order used in the live demonstration.

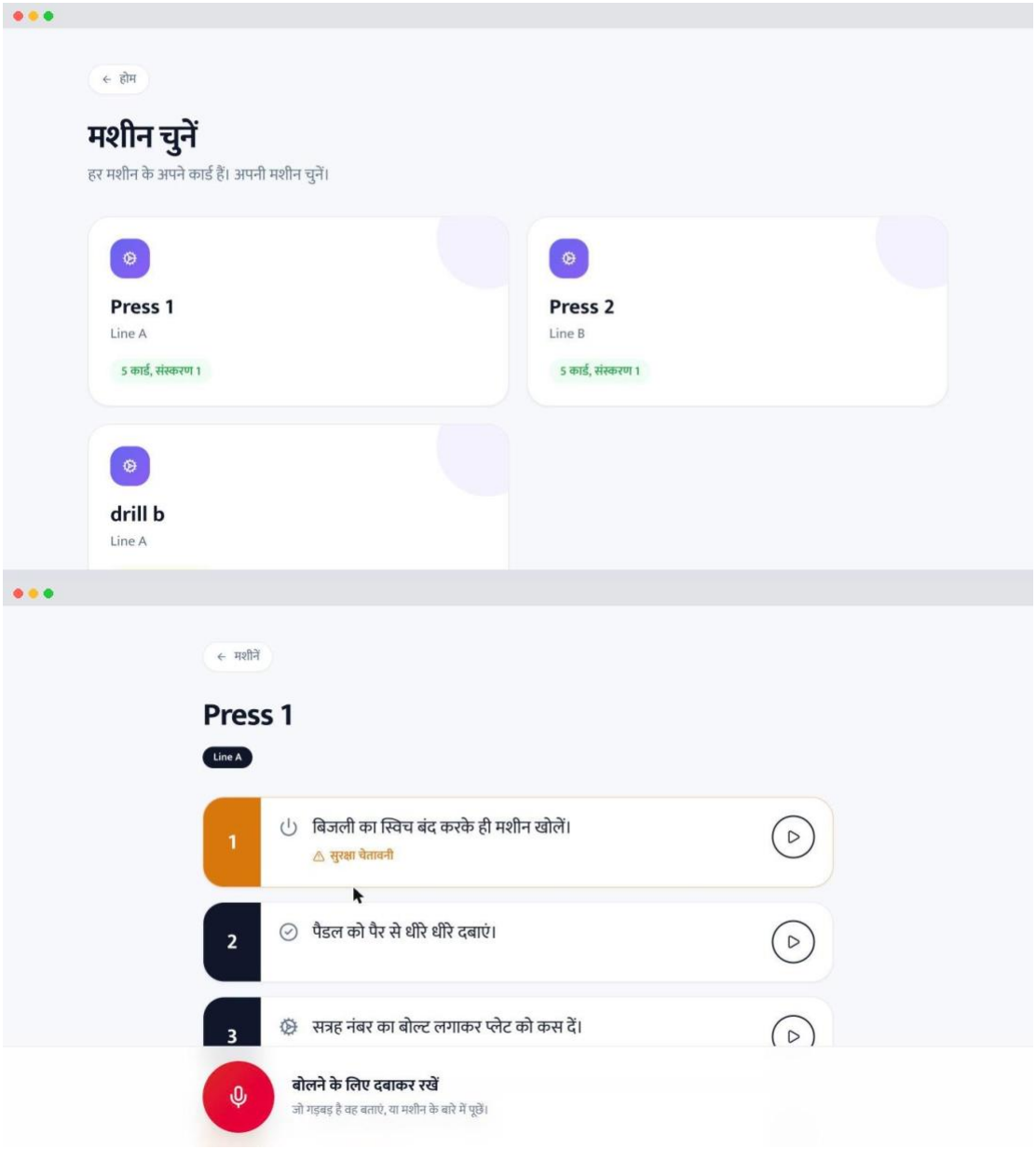


Figure 5.1: Worker view: machine list and SOP cards

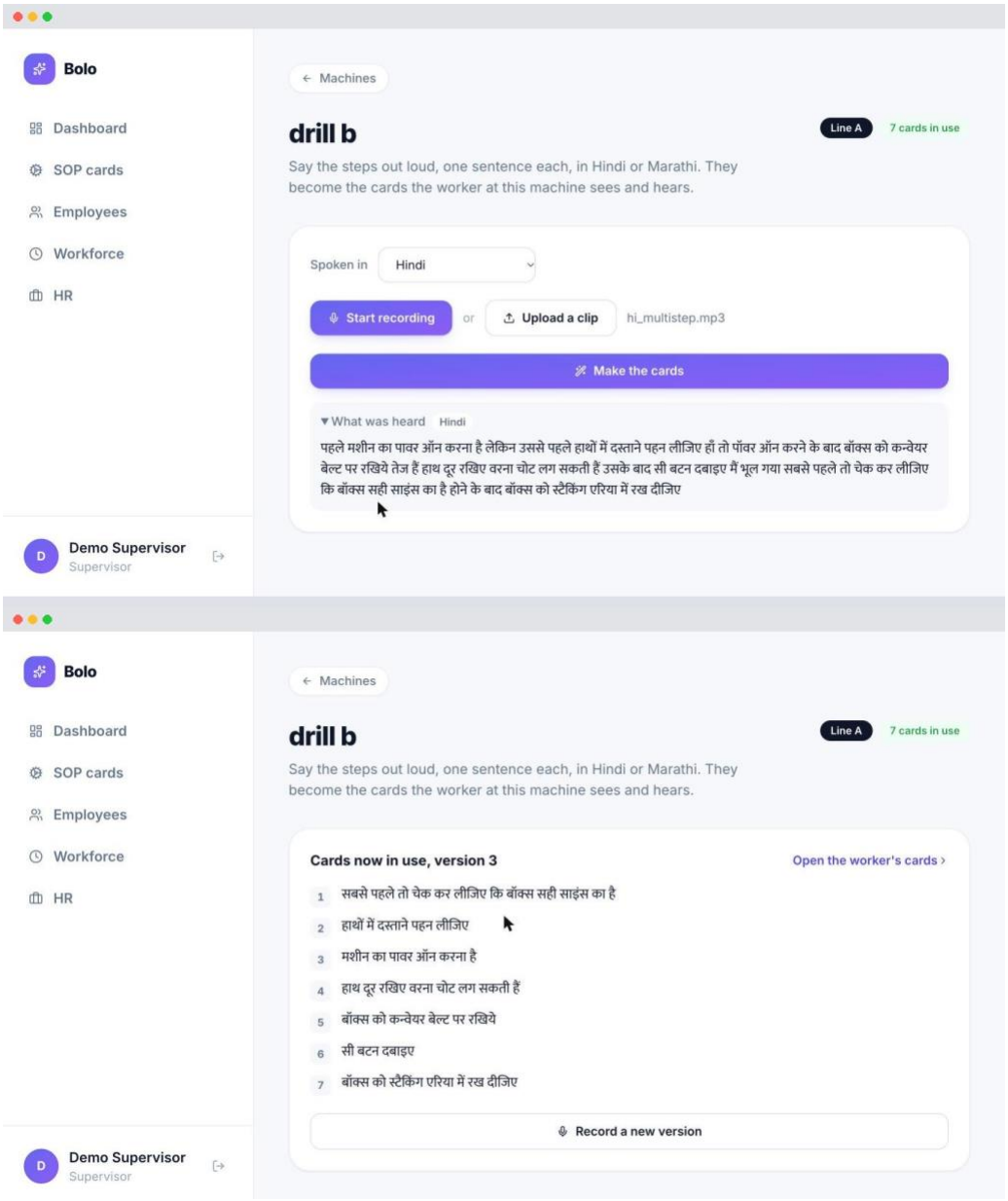


Figure 5.2: Supervisor view: recording and reviewing a procedure

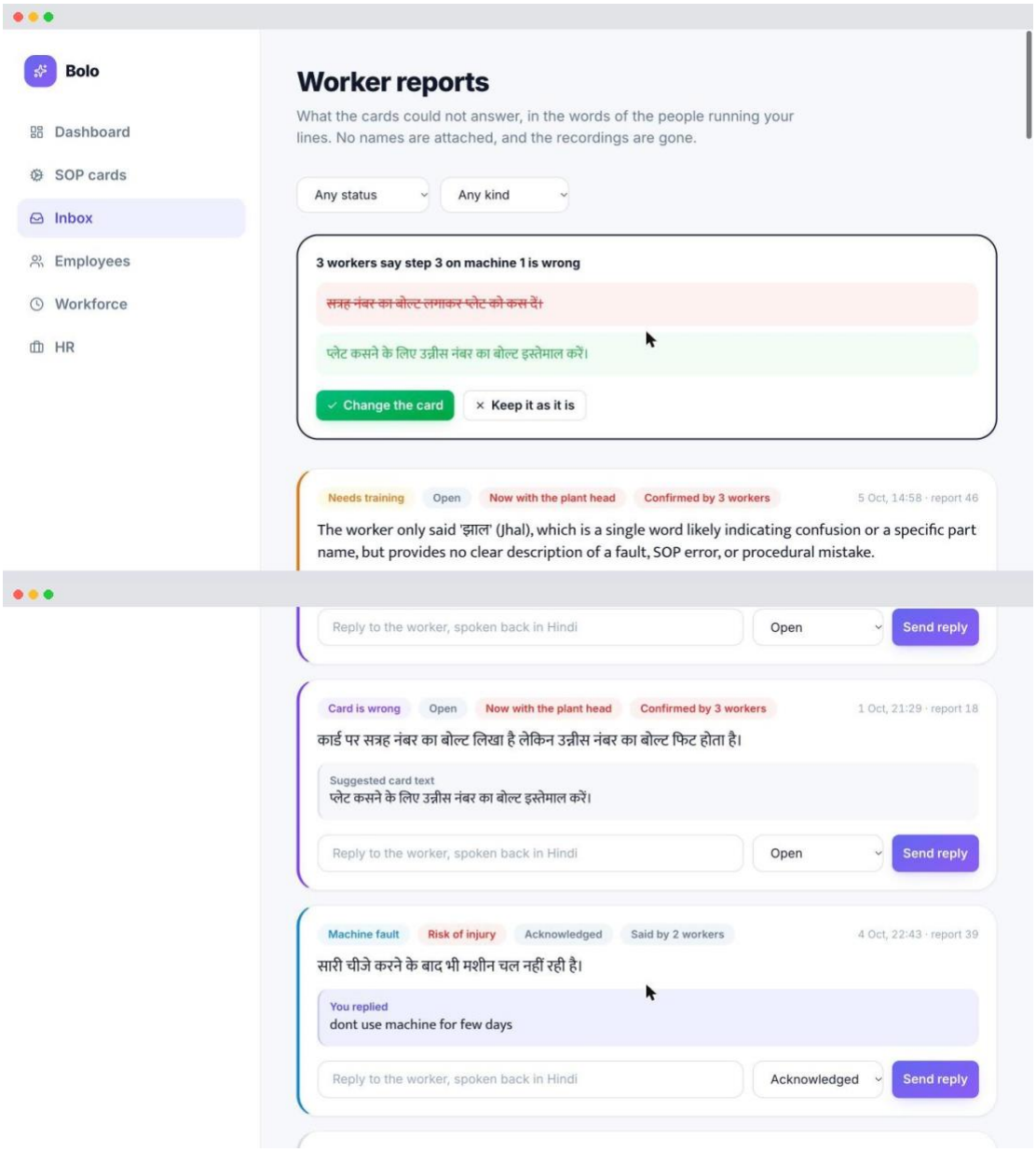


Figure 5.3: Manager view: triaged report inbox

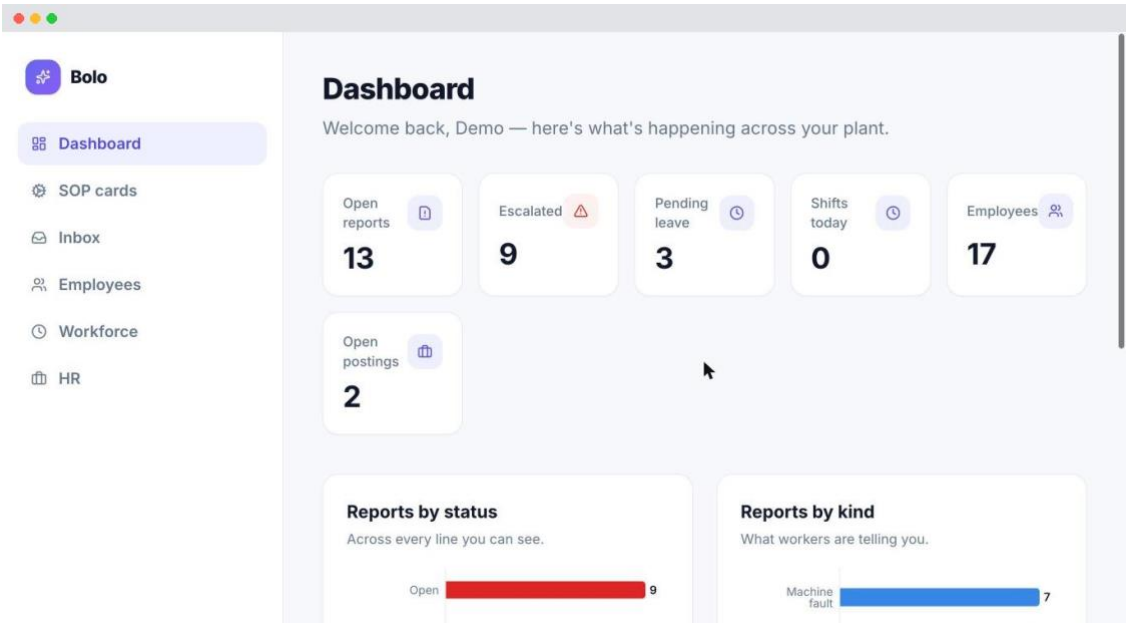


Figure 5.4: Plant head view: dashboard

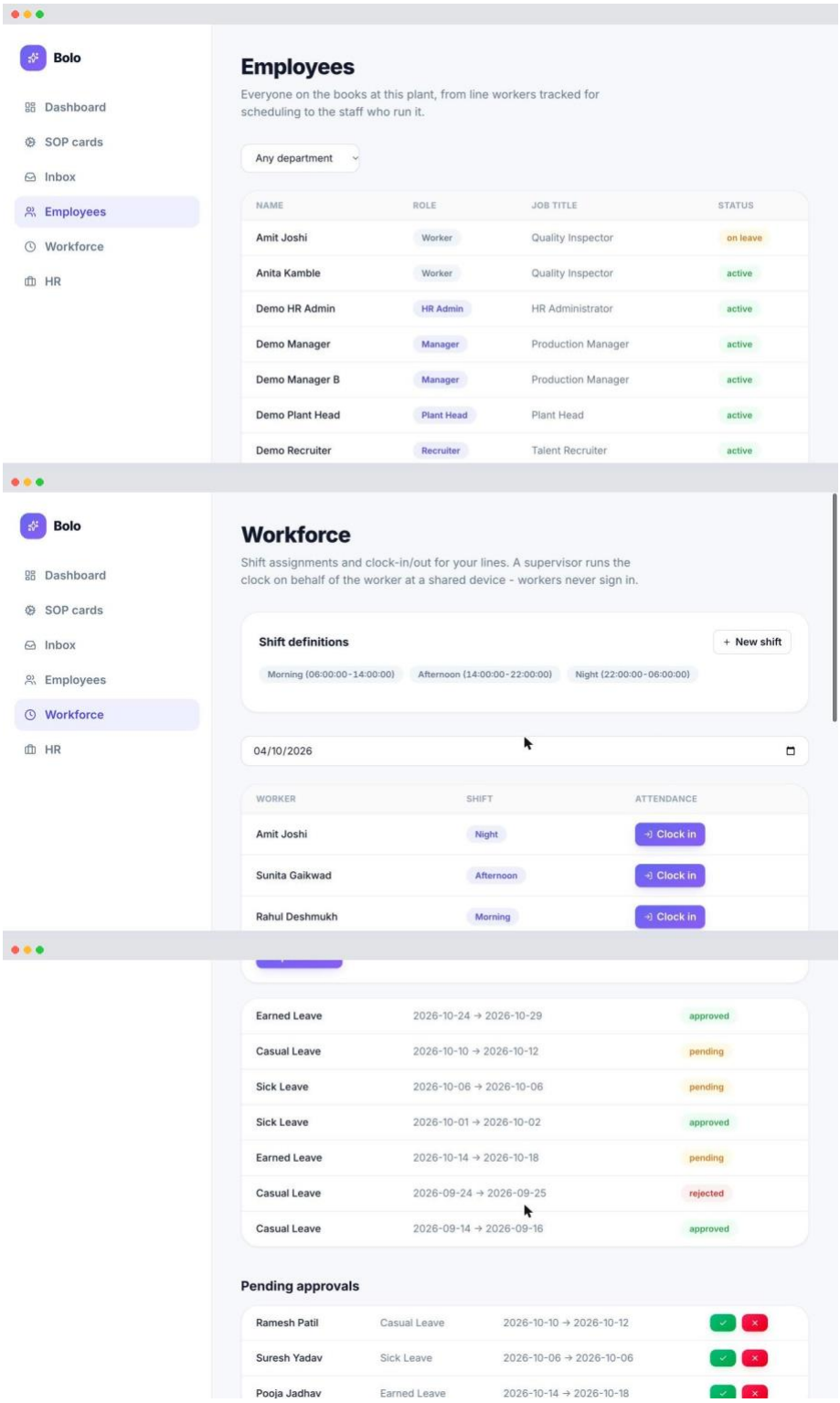


Figure 5.5: Employee, Workforce and HR modules

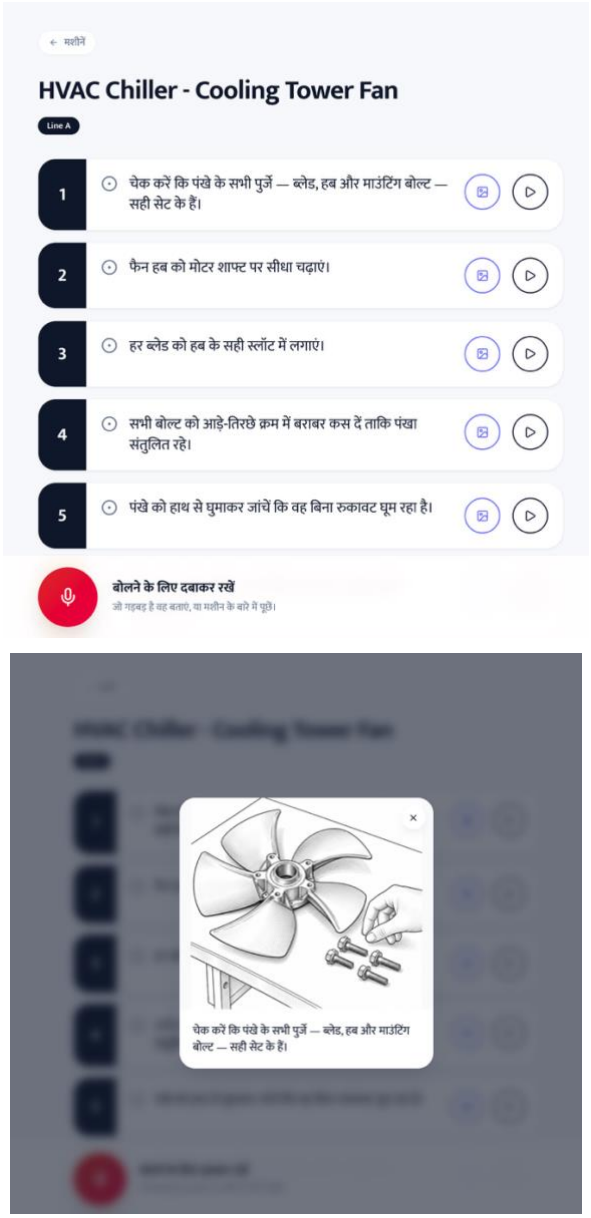


Figure 5.6: Worker view: voice report and receipt

5.4 Testing

The following types of testing were used:

- **Unit testing:** pure functions such as triage post-processing are tested with the LLM call replaced by a stub.
- **Integration testing:** routing, clustering and access rules are tested with pytest against a local Supabase Postgres stack that is truncated and reseeded with a fixed demo organisation for each test.
- **Security and access-control testing:** valid, tampered, expired and missing tokens, unknown profiles, and attempts to act outside one's own line.

- **End-to-end testing:** scripts/e2e.py runs the full checklist against a running server with real ASR, LLM and TTS, using clips synthesised with edge-tts.
- **Manual walkthrough and CI:** TESTING.md guides a hands-on run of the whole loop; GitHub Actions runs ruff and pytest on every push and pull request.

Table 5.2: Unit and integration test cases

ID	Test case	Expected result	Result
UT1	Triage returns a step id not in the SOP	step_id set to null	Pass
UT2	Triage returns a valid step id	step_id kept	Pass
UT3	Route a new report	Assigned to line manager, not supervisor	Pass
UT4	High-severity machine report vs other report	Deadline 24 h vs 72 h	Pass
UT5	Two open reports, same machine, kind and step	Linked into one cluster	Pass
UT6	Two reports of different kinds	Not clustered	Pass
UT7	New report matching a resolved one	Not clustered	Fail
UT8	Valid staff token	Resolves to the right person and role	Pass
UT9	Tampered or expired token	Rejected with 401	Pass
UT10	No token on a staff route	401, not 403	Pass
UT11	Valid session with no staff profile	Rejected	Pass
UT12	Worker machine list without a token	200 OK	Pass
UT13	Add machine without a token	Rejected	Pass
UT14	Supervisor adds machine to own line / other line	Allowed / 403	Pass
UT15	Plant head adds machine to any line in plant	Allowed	Pass

Table 5.3: End-to-end test cases

ID	Test case	Expected result	Result
E1	Narrated multi-sentence procedure	Every sentence transcribed	Pass
E2	Structure and save an SOP	Steps returned, saved and read back in order	Pass
E3	Worker uploads a report	Immediate receipt; no identity stored	Pass
E4	Machine, SOP and understanding complaints	Each triaged to the correct kind	Pass
E5	After processing a report	Report audio deleted from disk	Pass
E6	Question answered by the SOP	Answered aloud; invisible to manager (404)	Pass
E7	Manager replies to a report	Reply stored, voiced, playable via receipt	Pass

ID	Test case	Expected result	Result
E8	Report left open past its deadline	Escalates to the plant head	Pass
E9	Three similar reports	One confirmed cluster	Pass
E10	Supervisor / other line's manager reads report	403 / 404	Pass
E11	Confirmed SOP cluster, then approval	Step edit drafted; version bumped, card changed	Pass

5.5 Discussion

The results show that the pipeline turns unordered spoken Hindi into ordered, hazard-flagged cards, and that the worker loop closes in both directions: a report reaches the line manager with a deadline, an ignored report escalates, repeated card complaints change the card, and the worker hears the outcome. Questions the SOP already answers never reach the manager, which keeps the inbox focused on real problems.

Several limitations remain. Escalation is computed when the inbox is read, so nothing escalates until a manager opens it. Triage accuracy has not yet been measured on real factory recordings, and Marathi recognition is weaker than Hindi. Hiding the worker's name reduces, but does not remove, the chance that a manager guesses who spoke on a very small line. Rate limits are kept in memory per process, so a multi-process deployment needs a shared store such as Redis. Leave requests are available only to staff with accounts, and payroll is out of scope.

CHAPTER 6
CONCLUSION

6.1 Conclusion

This project set out to close two gaps on Indian shop floors: workers cannot read the SOPs written for them, and the hazards they see do not reach anyone who can fix them. In Stage I, the following work was completed:

- Voice-to-SOP was implemented end to end: Hindi and Marathi transcription with silence-split windows, LLM structuring into ordered steps with safety flags, and per-step vernacular audio on visual cards.
- Worker voice feedback (Bolo) was implemented: hold-to-talk reports, SOP-aware triage, skip-level routing with deadline-based escalation, clustering, auto-drafted card corrections, and answers from the SOP for questions it already covers.
- Worker identity is protected by design: no stored audio and no worker account, while staff access is scoped to their own lines and enforced again by Postgres Row Level Security.
- The backend was migrated to Supabase Postgres, Supabase Auth and Row Level Security, with every schema change as a hand-written Alembic migration.
- At the client's request, employee, workforce and HR modules were added on the same organisation structure and authorisation model, with a single role-filtered dashboard for all staff.
- The system ships as one Docker image with a pytest suite, an end-to-end verification script, structured logs and Prometheus metrics.

By catching near-misses at the bottom of the accident triangle and by guaranteeing that someone above the supervisor must act, the system aims to prevent injuries rather than record them afterwards.

6.2 Future Work

- Add a phone-call (IVR) channel so that workers with basic phones can report without the app.

- Validate Marathi and code-mixed Hindi-Marathi-English recognition, and measure triage accuracy, on real factory recordings.
- Pilot the system in a plant to measure reporting rates, response times and anonymity on small lines.
- Move rate limiting to a shared store (Redis) for multi-process deployments, and run escalation on a schedule as well as on read.
- Extend the HR module to payroll, for which the schema already leaves room.

REFERENCES

[1] K. S. Bhogale, S. Sundaresan, A. Raman, T. Javed, M. M. Khapra and P. Kumar, "Vistaar: Diverse Benchmarks and Training Sets for Indian Language ASR," in Proc. INTERSPEECH, 2023.

[2] "Whisper-Based Multilingual ASR for Indic Languages," in Proc. Pattern Recognition and Machine Intelligence (PReMI), Springer LNCS, doi: 10.1007/978-3-032-18480-1_78.

[3] "Hindi-Marathi Code-Switched Speech Recognition," Preprints.org, manuscript 202503.2278, 2025.

[4] D. Y.-B. Wang, Z. Shen, S. S. Mishra, Z. Xu, Y. Teng and H. Ding, "SLOT: Structuring the Output of Large Language Models," arXiv:2505.04016, 2025.

[5] F. Longo and A. Padovano, "Voice-enabled Assistants of the Operator 4.0 in the Social Smart Factory," Manufacturing Letters, vol. 26, pp. 12-16, 2020.

[6] T. M. Probst, "Organizational Safety Climate and Supervisor Safety Enforcement: Multilevel Explorations of the Causes of Accident Underreporting," Journal of Applied Psychology, vol. 100, pp. 1899-1907, 2015, doi: 10.1037/a0039195.

[7] J. R. Detert and L. K. Treviño, "Speaking Up to Higher-Ups: How Supervisors and Skip-Level Leaders Influence Employee Voice," Organization Science, vol. 21, no. 1, pp. 249-270, 2010, doi: 10.1287/orsc.1080.0405.

[8] P. Mudliar, J. Donner and W. Thies, "Emergent Practices Around CGNet Swara, a Voice Forum for Citizen Journalism in Rural India," in Proc. Int. Conf. on Information and Communication Technologies and Development (ICTD), ACM, 2012.

[9] A. J.-P. Tixier, M. R. Hallowell, B. Rajagopalan and D. Bowman, "Automated Content Analysis for Construction Safety: A Natural Language Processing System to Extract Precursors and Outcomes from Unstructured Injury Reports," Automation in Construction, vol. 62, pp. 45-56, 2016.

[10] A. Radford, J. W. Kim, T. Xu, G. Brockman, C. McLeavey and I. Sutskever, "Robust Speech Recognition via Large-Scale Weak Supervision," in Proc. ICML, 2023.

[11] Safe in India Foundation, "CRUSHED2026: Safe in India's 8th Annual Report on Worker Safety in the Indian Automotive Manufacturing Sector," 2026. [Online]. Available: https://www.safeinindia.org/worker-safety-reports

[12] IndiaSpend, "Three Workers Die Every Day in Indian Factories, Govt Data Show," DGFASLI data obtained under RTI, republished by Janata Weekly. [Online]. Available: https://janataweekly.org/three-workers-die-every-day-in-indian-factories-govt-data-show/

[13] IndiaSpend / Newslaundry, "Safety Rules Are Routinely Flouted in India's Factories," 15 June 2026, reporting Safe in India's CRUSHED2026 findings. [Online]. Available: https://www.newslaundry.com/amp/story/2026%2F06%2F15%2Fsafety-rules-are-routinely-flouted-in-indias-factories

[14] Office of the Registrar General & Census Commissioner, India, "Census of India 2011: Language Data," 2011; summarised in "Multilingualism in India," Wikipedia. [Online]. Available: https://en.wikipedia.org/wiki/Multilingualism_in_India

[15] F. E. Bird Jr., accident-ratio study of 1.7 million accident reports from about 300 companies, 1966; summarised in "Accident triangle," Wikipedia. [Online]. Available: https://en.wikipedia.org/wiki/Accident_triangle

[16] T. Javed et al., "IndicSUPERB: A Speech Processing Universal Performance Benchmark for Indian Languages," in Proc. AAAI Conference on Artificial Intelligence, 2023 (Kathbath dataset).

[17] A. Diwan et al., "MUCS 2021: Multilingual and Code-Switching ASR Challenges for Low Resource Indian Languag," in Proc. INTERSPEECH, 2